

Priority Queues: Introduction

Data Structures

Data Structures and Algorithms

Queue



A **queue** is an abstract data type supporting the following main operations:

- `PushBack(e)` adds an element to the back of the queue;
- `PopFront()` extracts an element from the front of the queue.

Priority Queue (Informally)

A **priority queue** is a generalization of a queue where each element is assigned a priority and elements come out in order by priority.

Priority Queues: Typical Use Case

Scheduling jobs

- Want to process jobs one by one in order of decreasing priority. While the current job is processed, new jobs may arrive.

Priority Queues: Typical Use Case

Scheduling jobs

- Want to process jobs one by one in order of decreasing priority. While the current job is processed, new jobs may arrive.
- To add a job to the set of scheduled jobs, call `Insert(job)`.

Priority Queues: Typical Use Case

Scheduling jobs

- Want to process jobs one by one in order of decreasing priority. While the current job is processed, new jobs may arrive.
- To add a job to the set of scheduled jobs, call `Insert(job)`.
- To process a job with the highest priority, get it by calling `ExtractMax()`.

Priority Queue (Formally)

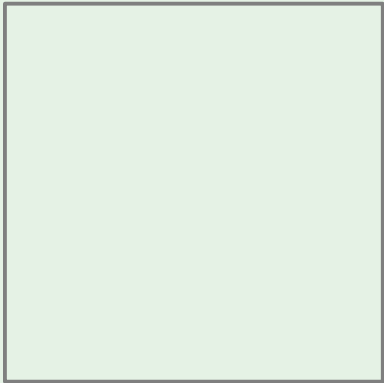
Definition

Priority queue is an abstract data type supporting the following main operations:

- `Insert( $p$ )` adds a new element with priority p
- `ExtractMax()` extracts an element with maximum priority

Example

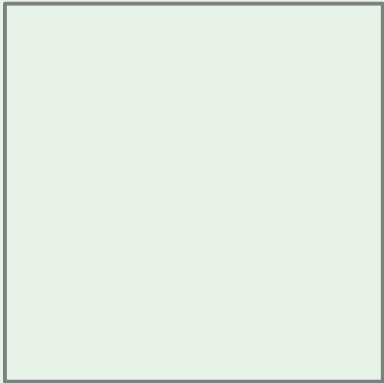
Contents:



Queries:

Example

Contents:



Queries:

Insert(5)

Example

Contents:

5

Queries:

Example

Contents:

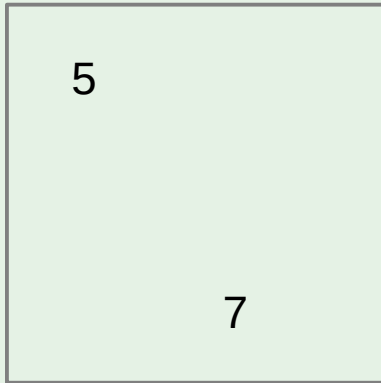
5

Queries:

Insert(7)

Example

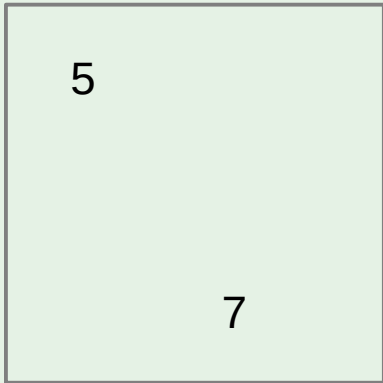
Contents:



Queries:

Example

Contents:

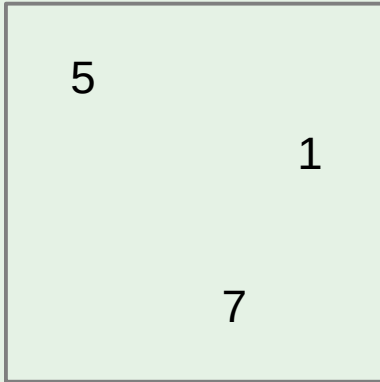


Queries:

Insert(1)

Example

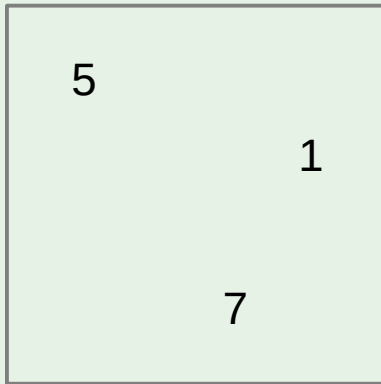
Contents:



Queries:

Example

Contents:

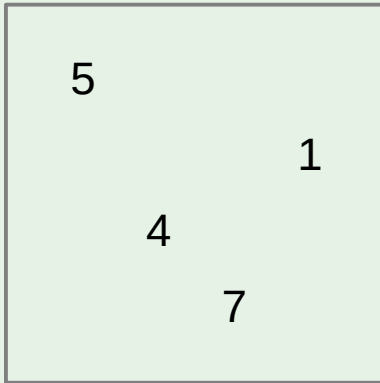


Queries:

Insert(4)

Example

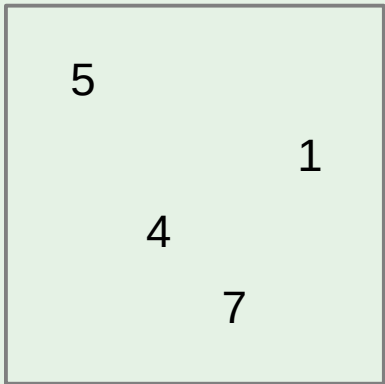
Contents:



Queries:

Example

Contents:

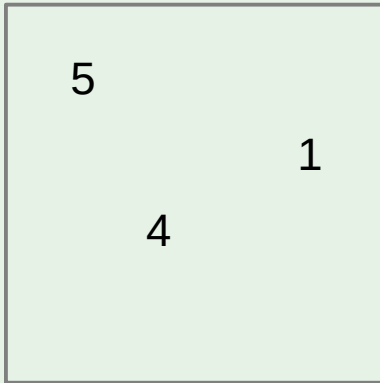


Queries:

`ExtractMax()` $\rightarrow$ 7

Example

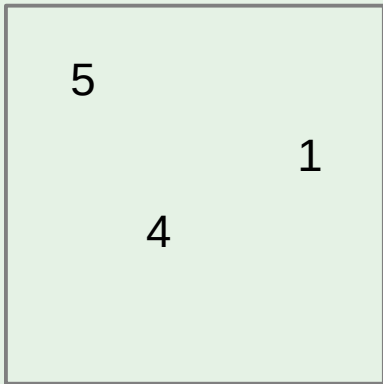
Contents:



Queries:

Example

Contents:

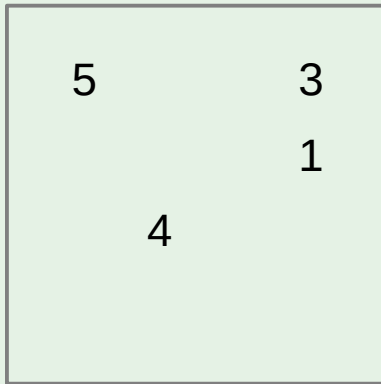


Queries:

Insert(3)

Example

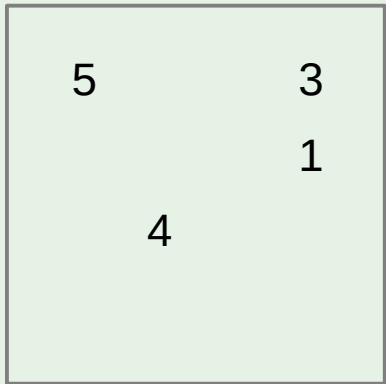
Contents:



Queries:

Example

Contents:

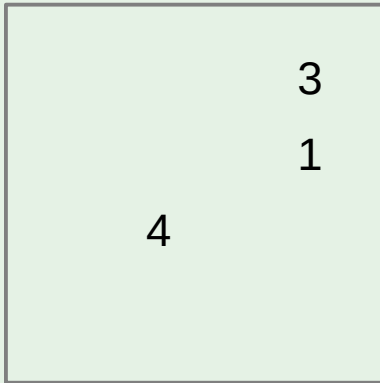


Queries:

`ExtractMax()` $\rightarrow$ 5

Example

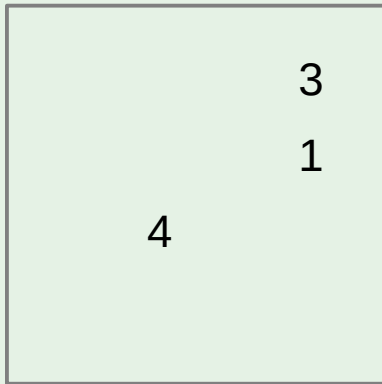
Contents:



Queries:

Example

Contents:

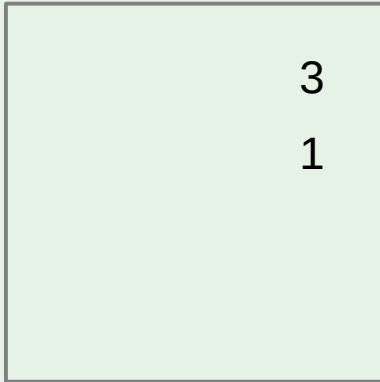


Queries:

`ExtractMax()` $\rightarrow$ 4

Example

Contents:



Queries:

Question

What will be the output of the following program? (As an answer, provide a sequence of integers separated by spaces.)

create an empty priority queue

Insert(18)

Insert(12)

Insert(14)

print(ExtractMax())

print(ExtractMax())

Insert(15)

print(ExtractMax())

Insert(10)

print(ExtractMax())

print(ExtractMax())

18,14,15,12,10

Additional Operations

- `Remove(it)` removes an element pointed by an iterator *it*
- `GetMax()` returns an element with maximum priority (without changing the set of elements)
- `ChangePriority(it, p)` changes the priority of an element pointed by *it* to *p*

**Let's take detour and
learn binary search first**

Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq \text{value} \leq a[hi]$.

Ex. Binary search for 33.

[illegible]

Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.

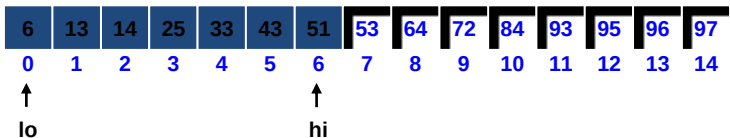
6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑							↑							↑
lo							mid							hi

Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.



If the value at `mid` is greater than the target, `hi` is updated to `mid - 1`.
If the value at `mid` is less than the target, `lo` is updated to `mid + 1`.

Binary Search

Binary search. Given value and sorted array $a[]$, find index i such that $a[i] = \text{value}$, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq \text{value} \leq a[hi]$.

Ex. Binary search for 33.

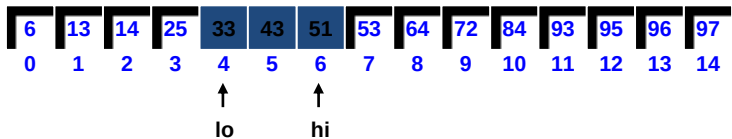
6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑			↑			↑								
lo			mid			hi								

Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.

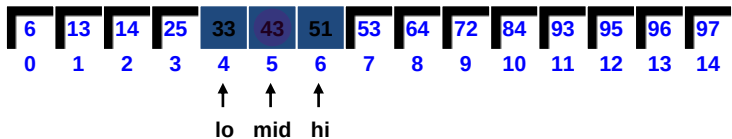


Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.

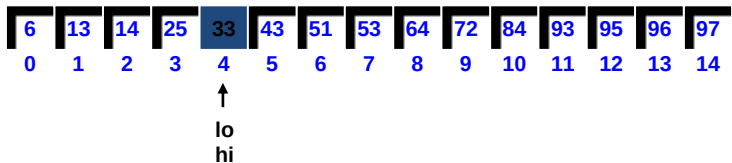


Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.

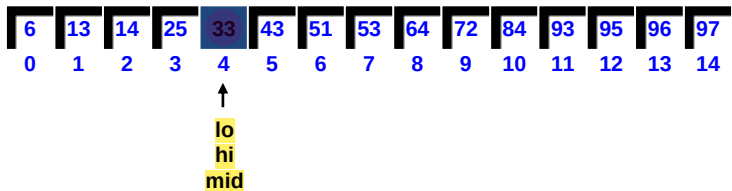


Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.

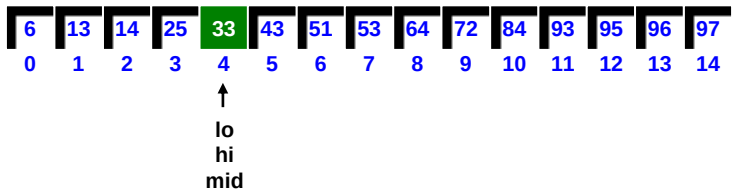


Binary Search

Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

Ex. Binary search for 33.



Time Complexity of the Binary Search

- Initial length of array = n
- Iteration 1 - Length of array = $n/2$
- Iteration 2 - Length of array = $(n/2)/2 = n/2^2$
- Iteration k - Length of array = $n/2^k$

After k iterations, the size of the array becomes 1 (narrowed down to the first element or last element only).

$$\text{Length of array} = n/2^k = 1$$

$$\Rightarrow n = 2^k$$

Applying log function on both sides:

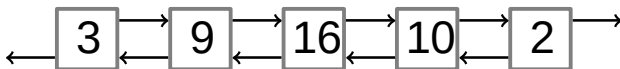
$$\Rightarrow \log_2(n) = \log_2(2^k)$$

$$\Rightarrow \log_2(n) = k * \log_2 2 = k$$

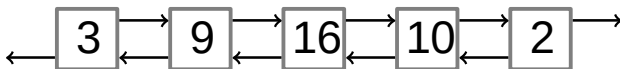
$$\Rightarrow k = \log_2(n)$$

Priority Queue Implementation

Unsorted Array/List

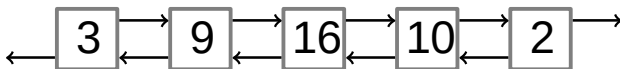


Unsorted Array/List



- Insert(e)
 - add e to the end
 - running time: $O(1)$

Unsorted Array/List



- `Insert(e)`
 - add *e* to the end
 - running time: $O(1)$
- `ExtractMax()`
 - scan the array/list
 - running time: $O(n)$

Sorted Array

2	3	9	10	16				
---	---	---	----	----	--	--	--	--

Sorted Array

2	3	9	10	16				
---	---	---	----	----	--	--	--	--

- `ExtractMax()`
 - extract the last element
 - running time: $O(1)$

Sorted Array

2	3	9	10	16				
---	---	---	----	----	--	--	--	--

- `ExtractMax()`
 - extract the last element
 - running time: $O(1)$
- `Insert(e)`
 - find a position for *e* ($O(\log n)$) by using **binary search**, shift all elements to the right of it by 1 ($O(n)$), insert *e* ($O(1)$)
 - running time: $O(n)$

Sorted List



Sorted List



- `ExtractMax()`
 - extract the last element
 - running time: $O(1)$

Sorted List



- `ExtractMax()`
 - extract the last element
 - running time: $O(1)$
- `Insert(e)`
 - find a position for *e* ($O(n)$; note: cannot use binary search), insert *e* ($O(1)$)
 - running time: $O(n)$

Question

Assume that you know in advance that in your application there will be n calls to Insert and $\sqrt{n}$ calls to ExtractMax. Which of the following two implementations of the priority queue is preferable in this case? Explain your response.

- ☐ Array
- ☐ Sorted array

Summary

	Insert	ExtractMax
Unsorted array/list	$O(1)$	$O(n)$
Sorted array/list	$O(n)$	$O(1)$

Summary

	Insert	ExtractMax
Unsorted array/list	$O(1)$	$O(n)$
Sorted array/list	$O(n)$	$O(1)$
Binary heap	$O(\log n)$	$O(\log n)$

Review Question

```
FUNCTION binary_search(arr, target):
```

```
  left ← 0
```

```
  right ← LENGTH(arr) - 1
```

```
  WHILE left ≤ right:
```

```
    mid ← left + (right - left) // 2
```

```
    IF arr[mid] = target THEN
```

```
      RETURN mid
```

```
    ELSE IF arr[mid] < target THEN
```

```
      left ← mid + 1
```

```
    ELSE
```

```
      right ← mid - 1
```

```
  RETURN -1
```

1. What is the purpose of the mid variable in the binary search algorithm, and how is it calculated?
2. Explain why the condition $\text{left} \leq \text{right}$ is necessary for the loop to continue executing.
3. Derive the time complexity of the binary search algorithm in terms of the number of elements in the array. What is the significance of this time complexity compared to a linear search algorithm?